



← Back to student guides

[📈] MLflow

Tracking & registry

3 levels

111 sections



The Complete Guide

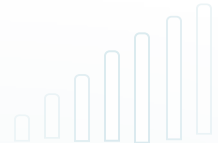
Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

RUN

MODEL

REGISTRY

SERVE



Quick Start

A one-page starter for this tool. Scroll through it here, or open it full screen.

Open PDF







Part three of three. The problems at this level are rarely about syntax. They are about a server thirty people depend on, an artifact bill nobody owns, a champion nobody can rebuild, and a promotion nobody can explain. Start with the error table, then the practices, verification, and playbooks underneath it.

Common errors at this level

Cumulative. Everything from Beginner and Mid still applies. These cause incidents rather than failed runs.

Symptom	Real cause	Fix
A researcher promoted a model to production	Anyone with tracking credentials can move an alias	<code>READ</code> on registered models; alias changes only from a gated CI job
Nobody can say who promoted the champion	Promotion was a click, not a job	A promotion script that writes an audit tag, run from a reviewed workflow
An unauthenticated server was reachable from the office network	No auth app, no proxy	TLS + auth at a gateway; MLflow's basic-auth plugin at minimum
Every user and CI job holds bucket credentials	Default direct artifact access	<code>--serve-artifacts --artifacts-destination</code>
An "immutable" version served different weights	Someone overwrote the object behind it	Write-restricted prefix, bucket versioning, proxied access
A restored database resolved every model and downloaded none	Backup covered metadata only	The bucket is part of the backup; verify a download on restore



database is locked under load	SQLite in a shared deployment	Postgres or MySQL
A container restart migrated the schema	Server image on <code>latest</code>	Pin the tag; run <code>mlflow db upgrade</code> deliberately
All logging and the UI stopped at once	Backend database full	Disk alert at 80%, retention, <code>mlflow gc</code> , per-epoch logging
Production scoring failed during a database blip	Serving resolved the alias per request	Resolve once at startup and log the version
A batch job failed at 3am with a missing artifact	A lifecycle rule expired a registered version's object	Registry prefix exempt from expiry; verify rules before enabling
Deleting a thousand runs freed nothing	<code>delete_run</code> is a soft delete	<code>mlflow gc --older-than</code>
Retention deleted the evidence for the champion	No exemption for registry-referenced runs	Exempt them, and <code>keep</code> -tagged runs, and dry-run first
A promotion broke every consumer	Signature changed; an alias move skips code review	Additive-only signatures, or a new registered model name
The champion cannot be rebuilt	Dirty tree, unpinned base image, or no dataset recorded	Dirty-tree guard, required <code>log_input</code> , image by digest
A rebuild reproduced but the metric drifted	Unpinned wheels or non-determinism	Hash-pinned serving requirements; a tolerance in the check
An audit could not name the training data	The run read a raw path	Require a dataset input; fail the run without one
Customer identifiers found in run artifacts	Scored predictions logged as artifacts	Classify artifacts; strip identifiers before logging



Real training rows inside a published model	<code>log_input_examples=true</code> under autolog	Turn it on, or use a synthetic example
A trace store full of user messages	Tracing enabled on production traffic with no policy	Redaction and retention decided before enabling
Artifact storage tripled with no new projects	A model logged per sweep child	Log models for the winner only; lifecycle per prefix
Cost could not be attributed to a team	No prefixes, no mandatory tags	Design attribution in; it cannot be reconstructed later
A fork pull request wrote to the registry	The PR credential could register and alias	PR credentials create runs only; promotion on protected branches

The practices that pay off most



Alias write belongs to one job

Researchers register; a gated CI account promotes. This is the single highest-value control in MLflow.



Proxied artifacts

`--serve-artifacts` so only the server holds bucket keys and access follows MLflow permissions.



Bucket versioning + write restriction

Together they make "version 7" mean version 7, and a bad delete survivable.



Alert on database disk first

When it fills, logging, the UI, and alias resolution stop together. One alert prevents the worst outage.



Restore, do not just back up

Database and bucket, rehearsed quarterly, with a published RTO, and verify a model download.

Digests, not tags

Base image by digest plus hash-pinned serving requirements. The difference between reproducible and probably.



Weekly rebuild of the champion

The only mechanism that turns reproducibility from a claim into a monitored property.



A wrapper, not a wiki

Experiment path, tags, registered name, requirements, guards: all defaults. The only governance that survives a deadline.

Practice cards

01

Test the promotion boundary

With a researcher credential, try to move a production alias. If it works, stop and fix that before anything else on this page.

02

Replace a version's bytes

On a test bucket, overwrite the object behind a registered version and load it. Nothing warns you. That is the demonstration.

03

Restore and time it

Database and bucket onto a scratch host. Verify login, metrics, alias resolution, **a model download**, and a new run. Publish the number.

04

Fill the database



05

Prove the serving decoupling

Stop the tracking server while a batch job runs. If it dies, you resolve the alias per request.

06

Rebuild the champion

From only what MLflow recorded. Compare the metric, then list every layer that run does not pin.

07

Generate a lineage report

Code, data, config, and approval in one command. Note which of the four is missing.

08

Audit artifacts for identifiers

Search prediction files and embedded input examples. Decide what is in scope for erasure.

09

Dry-run retention

Read the list, the bytes it would free, and confirm the registry-reference exemption works.

10

Run the checklist on someone else's project

It will find something, and it teaches you how much of this is convention rather than enforcement.

The hardening pass every deployment should get

Work down this list. Each step is independently valuable, and the order is by blast radius.

BASH

Copy

```
# 1. Alias permissions – the production-change surface
# Researchers: EDIT on experiments, READ on registered models.
```



```
auth = get_app_client("basic-auth", tracking_uri="https://mlflow.internal")
auth.create_registered_model_permission(name="risk-churn-classifier",
                                       username="amina", permission="READ")
auth.create_registered_model_permission(name="risk-churn-classifier",
                                       username="ci-promote", permission="EDIT")
```

PY

2. Auth and TLS in front of the server

```
mlflow server --backend-store-uri "$PG" \
  --artifacts-destination s3://ml-artifacts/mlflow --serve-artifacts \
  --app-name basic-auth --host 127.0.0.1 --port 5000 --workers 4
```

3. Bucket safety – makes every other storage mistake survivable

```
aws s3api put-bucket-versioning --bucket ml-artifacts \
  --versioning-configuration Status=Enabled
```

4. Postgres, not SQLite; pinned server image, not latest

ghcr.io/mlflow/mlflow:v2.16.0 + mlflow==2.16.0 clients

5. Alerts before anything else

```
# backend database disk > 80%    → page
# database connections > 80%     → page
# server 5xx sustained           → page
```

lifecycle rules, per prefix

Copy

```
s3://ml-artifacts/<team>/research/ 30d → IA, 180d → Glacier, 400d → delete
s3://ml-artifacts/<team>/ci/        14d → delete
s3://ml-artifacts/<team>/registry/  NO EXPIRY – a registered version points here
```

6. retention, dry-run gated and registry-aware

Copy

```
APPLY = os.environ.get("RETENTION_APPLY") == "1"
EXEMPT_TAGS = {"keep", "paper", "release"}
# skip anything a registered version references; print totals; delete only when enabled
```

BASH

Copy



Step	Prevents
Alias permissions	Anyone changing production, with no record
Auth and TLS	Credential interception and anonymous writes
Bucket versioning	Unrecoverable delete or silent overwrite
Postgres and a pinned image	Lock-ups, and unplanned schema migrations
Disk and connection alerts	The outage that stops all logging at once
Registry-exempt lifecycle	A batch job failing on a missing champion artifact
Gated retention plus gc	Deleting evidence, and reclaiming nothing

Do step one today

Everything else on this list protects against accident. Alias permissions protect production from anyone with credentials, and on a default install that is every person and every job that can reach the server. It is a ten-minute change with the largest blast radius on the page.

Verifying, not assuming

Every claim here is checkable in minutes. Assumed properties are the ones that fail.

Claim	How to verify
"Only CI can promote"	Try to move an alias with a researcher credential
"We have backups"	Restore onto a scratch host and download a model



The registry is authoritative	Resolve <code>@champion</code> and compare with what is serving
"Versions are immutable"	Overwrite the object behind one on a test bucket and load it
"We are reproducible"	Rebuild the champion and compare within a tolerance
"Serving survives a tracking outage"	Stop the server while a batch job runs
"Retention is safe"	Dry-run it and check the registry-reference exemption
"Artifacts are governed"	Confirm whether clients hold bucket keys
"The database has headroom"	Disk, connections, and row counts on the metrics table
"Promotion is auditable"	Ask someone else to reconstruct the last one from the registry alone

the five-minute platform verification

Copy

```
import json
import mlflow
from mlflow import MlflowClient

NAME = "risk-churn-classifier"
client = MlflowClient()

champion = client.get_model_version_by_alias(NAME, "champion")
run = client.get_run(champion.run_id)

assert run.data.tags.get("git_clean") == "True", "champion trained from a dirty tree"
assert run.inputs.dataset_inputs, "champion has no recorded dataset"
assert "promotion_audit" in champion.tags, "no approval record on the champion"
assert client.get_model_version_by_alias(NAME, "previous"), "no rollback target"

audit = json.loads(champion.tags["promotion_audit"])
print("version:", champion.version)
print("commit:", run.data.tags.get("mlflow.source.git.commit"))
print("data:", [d.dataset.source for d in run.inputs.dataset_inputs])
```



BASH

Copy

```
curl -sf "$MLFLOW_TRACKING_URI/health" && echo "server ok"
psql "$PG" -c "select pg_size_pretty(pg_database_size(current_database()));"
psql "$PG" -c "select count(*) from metrics;"
psql "$PG" -c "select count(*) from runs where lifecycle_stage='deleted';"
aws s3api get-bucket-versioning --bucket ml-artifacts
docker ps --format '{{.Names}}\t{{.Image}}' | grep mlflow      # is the tag pinned?
```

Turn the assertions into a scheduled run

Those four assertions are a continuously-verified statement about your production model: clean tree, recorded data, an approval record, and a rollback target. Run them weekly as a tracked run and you find out when someone breaks one of those properties. Instead of finding out during an audit.

Cost governance

Cost becomes your problem the moment MLflow is shared, and it is a measurement problem before it is a technical one.

Question	Answer it with
What is filling the artifact bucket?	A bucket inventory report, bytes by prefix, then by tag
How much of that is sweep children?	Runs with a <code>parentRunId</code> that logged a model
Why is the database large?	Row counts on <code>metrics</code> , and runs per experiment
How much is soft-deleted but unreclaimed?	<code>runs where lifecycle_stage='deleted'</code> , then <code>mlflow gc</code>



which team spent it:	mandatory <code>team</code> and <code>cost_centre</code> tags plus prefixes
What would lifecycle save?	Bytes older than each tier boundary, per prefix

platform/cost_report.py: weekly, itself a tracked run

Copy

```
import mlflow
from collections import defaultdict

by_team = defaultdict(int)
by_origin = defaultdict(int)
models_logged = defaultdict(int)

for exp in mlflow.search_experiments():
    frame = mlflow.search_runs(experiment_ids=[exp.experiment_id],
                              max_results=50_000, output_format="pandas")

    if frame.empty:
        continue
    team = frame.get("tags.team", "untagged")
    for value in (team if hasattr(team, "__iter__") else [team]):
        by_team[value] += 1
    if "tags.origin" in frame:
        for value in frame["tags.origin"].fillna("manual"):
            by_origin[value] += 1

logger = mlflow.start_run(run_name="cost report", tags={"origin": "platform"})
for team, count in by_team.items():
    mlflow.log_metric(f"runs::{team}", count)
for origin, count in by_origin.items():
    mlflow.log_metric(f"runs_by_origin::{origin}", count)
mlflow.end_run()
```

Lever	Typical saving	Cost of using it
Log models for the sweep winner only	Often the largest single win	You cannot resurrect a losing candidate's weights
Lifecycle rules per prefix	Steady and compounding	Older artifacts become slow or gone to retrieve



Per-epoch metric logging	Database growth and query speed	Less intra-epoch resolution
Retention plus mlflow gc	Reclaims what soft deletes did not	Irreversible once collected
Best + last checkpoints only	Large for deep learning	No mid-training forensics
Direct artifact access for huge models	Server throughput and cost	Clients need bucket credentials again

Do not tier storage without pricing retrieval

Glacier looks cheap until a rebuild check or an audit needs forty models back, and retrieval plus egress can exceed a year of standard storage. Keep anything the rebuild check or an auditor might need in a warm class, and model the read pattern rather than the shelf price.

Incident playbooks

Four incidents, each with a first action, a verification, and the prevention. Keep them in the repository.

The tracking server is gone.

01 Check whether serving survived

If it cached its model at startup, this is an inconvenience rather than an outage. Establish that first.

02 Stand up the same pinned image

A different version runs schema migrations on top of an incident.



04

Verify in order

Login → a known run's metrics → alias resolution → a model download → a new run logging.

05

Reconcile

Interrupted runs are stuck in `RUNNING` ; close them and record the metric tail that was lost.

The backend database is full. Everything stops together: logging, the UI, and alias resolution. Free space or fail over to a replica, then fix the cause, usually per-batch metric writes or no retention. Prevention: an 80% disk alert and `mlflow gc` on a schedule.

A model version's artifacts are unreadable. Roll `champion` back to `previous` first, then investigate: a bucket delete marker or overwrite, a lifecycle rule that expired the object, or an artifact root that changed after the run was created. Recover from bucket versioning if it is enabled, and if it is not, that is the finding.

The wrong model is live. Move `champion` to `previous` immediately. Then read the version's `promotion_audit` tag to establish what happened, and check whether the promoter should have been able to do it. If a researcher moved it, this is an access-control finding, not a mistake.

Incident	First action	Prevention
Server lost	Same pinned image, same database	Stateless server, tested restore, published RTO
Database full	Free space or fail over	Disk alert, retention, <code>mlflow gc</code> , per-epoch logging
Artifacts unreadable	Roll back, then check versioning and lifecycle	Versioning on, registry prefix exempt from expiry
Wrong model live	Roll back to <code>previous</code>	Alias permissions restricted to a gated job



Serving broken after promotion	Roll back, diff the signature	Additive-only signatures, validate before promoting
Champion unrebuildable	Run the lineage report	Dirty-tree guard, required dataset, weekly rebuild check

A playbook is only real once someone else has followed it

Have a person who did not build the platform work through one and write down where they got stuck. Those gaps (an unstated hostname, an assumed credential, a missing permission) are exactly the ones that matter at 2am.

Machine-learning specifics

A few things are MLflow-and-ML rather than general platform work, and they come up in senior interviews.

A signature is a published API. Because promotion is an alias move with no deploy, a renamed column or a tightened type reaches production without an API diff being reviewed. Additive-only, or a new registered model name and a deliberate consumer migration.

Aggregate metrics hide segment regressions. A model can improve overall AUC while getting materially worse for one customer group. A worst-segment threshold in the gate costs ten lines and catches exactly the regression that damages trust.

Checkpoint policy is a cost decision. Best plus last, not every epoch. Every-epoch checkpoints of a large model, logged as artifacts, dominate an artifact bill within a quarter and nobody ever loads epoch 47.

Sweep children should not each log a model. Forty children × 200 MB is 8 GB per sweep for candidates you will never deploy. Log models for the winner, keep the children's metrics.



your batch job.

Weights are the un-erasable artefact. Removing records from a dataset does not remove their influence, and for some model classes information is extractable. Retraining is the remedy, which is why the erasure conversation is architectural rather than a runbook.

Traces and input examples are the two data-exposure surfaces people miss.

`log_input_examples=True` embeds real rows inside a shareable artifact; tracing stores prompts and responses verbatim. Both are near-default settings that create copies nobody remembers during a request.

The one-sentence version of this whole page

Restrict who can change production, verify the properties you claim on a schedule, and make the safe path the shortest path through a wrapper and defaults. Everything else here is an instance of one of those three.

The checklist to run before shipping

Area	Check
Alias permissions	Can a researcher move a production alias? Is promotion gated and reviewed?
Auth	TLS and authentication in front of the server?
Artifact credentials	Proxied, or does every client hold bucket keys?
Bucket safety	Versioning on? Write-restricted prefixes? Registry prefix exempt from expiry?
Backend	Postgres or MySQL? Disk and connection alerts at 80%?



Backups	Database and bucket? Restored last quarter? Model download verified? RTO published?
Pinning	Server image tag pinned? Client/server aligned? Base images by digest?
Retention	Scheduled, dry-run gated, registry-aware? Is <code>mlflow gc</code> running?
Logging discipline	Per epoch, not per batch? No <code>search_runs</code> in loops?
Reproducibility	Dirty-tree guard? Dataset input required? Explicit serving requirements?
Rebuild check	Weekly, against the champion, with a tolerance, and green?
Lineage	Code, data, config, and approval for the champion in one command?
Promotion	Threshold-gated against a baseline, with an audit tag and a named reviewer?
Rollback	Is <code>previous</code> set at every promotion? Rehearsed?
Signatures	Additive-only policy? Validated in a built environment before promotion?
Serving	Authenticated? Alias resolved at startup? Resource limits?
Supply chain	Who can write the artifact bucket? Are external models reviewed before loading?
Data exposure	<code>log_input_examples</code> off? Prediction artifacts classified? Trace retention decided?
Cost	Storage growth published per team? Lifecycle verified against the registry?
Boundaries	Documented where MLflow stops and what owns orchestration, data, and serving?

Nobody passes this on the first attempt. Two failures matter disproportionately: **unrestricted alias permissions** and **an untested restore**: one means anyone can change production with no record, the other means a bad day becomes a permanent loss. Rank by blast radius rather than by effort, fix those two first, and publish the exceptions you are choosing to live with along with the reason. A checklist with argued exceptions is a better artefact than a green



PREVIOUS

[Senior · Interview Review](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Contact us

Privacy Policy

WHERE WE ARE

WhatsApp

Discord

LinkedIn

X

YouTube

GitHub

Zomra



Brainsmingle space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.